

Estrutura de Dados

Python Aplicado e Modelagem do Acervo

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Do Script ao Programa Organizado

Um script executa comandos de cima para baixo e cresce até que ninguém mais o entenda. Um programa organizado separa responsabilidades em funções e classes, cada uma com um propósito declarado. A diferença aparece na manutenção: no script toda alteração exige reler tudo, no programa organizado basta localizar a parte responsável.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Modelagem: Quem Guarda o Quê

A Faixa guarda os dados de uma música. Alguém precisa guardar o conjunto de faixas e responder por ele. Deixar essa responsabilidade espalhada pelo programa principal significa que qualquer trecho pode alterar a coleção sem controle. Concentrá-la num tipo próprio é o primeiro passo de qualquer modelagem honesta.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O TAD Acervo

O Acervo é o segundo tipo abstrato do projeto. Seus valores são as faixas cadastradas e suas operações são adicionar, buscar, remover, contar e percorrer. A lista do Python que guarda as faixas por dentro não faz parte do contrato: é apenas a implementação escolhida hoje, e será substituída adiante.



Prof. Everaldo Graciliano Souza Ribeiro

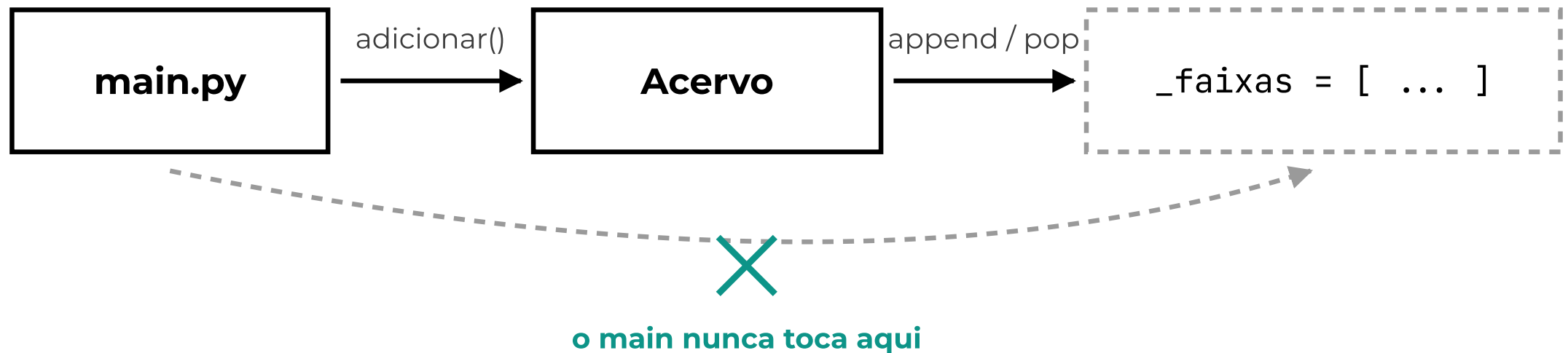
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Acervo Esconde a Lista



Trocar a lista por uma estrutura encadeada muda só o miolo do Acervo. **O main não fica sabendo.**



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Repetição e Condição a Serviço da Estrutura

Buscar um elemento é percorrer enquanto não encontrar. Remover é percorrer até localizar e então retirar. Toda operação de estrutura de dados se reduz a repetição combinada com condição, os mesmos recursos da lógica de programação já estudada. O que muda é a intenção: agora eles servem a um contrato.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Iterar sem Expor a Estrutura

Percorrer uma coleção não deveria exigir acesso à sua representação interna. Python resolve isso com métodos especiais: ao definir como o tipo é percorrido e como seu tamanho é obtido, o Acervo passa a ser usado com laços e contagem naturais, sem que ninguém precise conhecer a lista escondida por dentro dele.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Remover Custa Mais do que Inserir

Inserir no fim de uma lista não afeta os elementos existentes e tem custo constante. Remover de uma posição intermediária, ao contrário, obriga a deslocar todos os elementos seguintes uma casa para trás, a fim de fechar o buraco deixado. O custo dessa operação cresce com o tamanho da coleção.



Prof. Everaldo Graciliano Souza Ribeiro

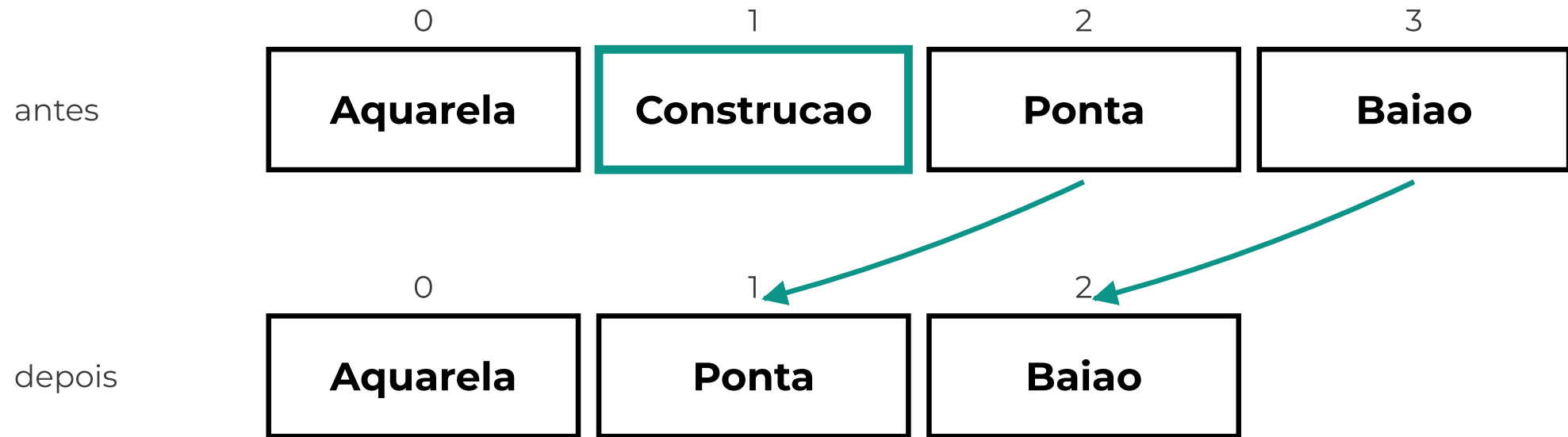
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Remover do Meio Desloca o Resto



2 deslocamentos aqui; com 4 000 faixas, quase 4 000

Inserir no fim é imediato. **Remover do meio desloca tudo o que vem depois.**



O Menu como Contrato com o Usuário

Um sistema só é útil quando alguém que não escreveu o código consegue operá-lo. O menu de console cumpre esse papel com o mínimo de recursos: apresenta as operações disponíveis, lê a escolha e a encaminha à função responsável. É a mesma separação de responsabilidades, agora entre interface e estrutura.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Acervo: Estado Interno

```
class Acervo:
    """Colecao de faixas cadastradas no SoundLab."""

    def __init__(self):
        # Representacao interna. O prefixo _ avisa: nao mexa daqui de fora.
        # Nas Aulas 04 e 05 esta lista vira uma estrutura encadeada.
        self._faixas = []

    def adicionar(self, faixa):
        """Insere no fim. Custo constante: nada e deslocado."""
        self._faixas.append(faixa)
```



Buscar e Remover

```
def buscar(self, titulo):  
    # Percorre posicao por posicao: e a busca linear.  
    for faixa in self._faixas:  
        if faixa.titulo.lower() == titulo.lower():  
            return faixa  
    return None  
  
def remover(self, titulo):  
    for posicao, faixa in enumerate(self._faixas):  
        if faixa.titulo.lower() == titulo.lower():  
            # pop(posicao) desloca UMA A UMA as faixas seguintes.  
            return self._faixas.pop(posicao)  
    return None
```



Contar e Percorrer

```
def __len__(self):  
    """Permite escrever len(acervo)."""  
    return len(self._faixas)  
  
def __iter__(self):  
    """Permite `for faixa in acervo` sem expor a lista interna."""  
    return iter(self._faixas)
```

```
# No main, o Acervo passa a ser usado como qualquer colecao do Python:  
for posicao, faixa in enumerate(acervo):  
    print(f" [{posicao}] {faixa}")
```



O Menu Principal

```
def main():
    acervo = carregar_acervo()
    # Cada opcao aponta para a funcao responsavel: sem cadeia de ifs.
    acoes = {"1": listar, "2": cadastrar, "3": buscar, "4": remover}

    while True:
        print(OPCOES)
        opcao = input(" opcao: ").strip()
        if opcao == "0":
            break
        acao = acoes.get(opcao)
        acao(acervo) if acao else print(" opcao invalida")
```



Atividades

Exercício 1

Por que o atributo da lista se chama `_faixas` e não `faixas`?

Exercício 2

Remover a primeira faixa de um acervo com 500 músicas desloca quantos elementos?
E remover a última?

Exercício 3

Que método precisa existir para que `for faixa in acervo` funcione?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Impedir o cadastro de duas faixas com o mesmo título.
2. Acrescentar ao menu uma opção que mostre a duração total do acervo.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>